DBI Mitschrift

20.10.2026 – Graphen, JSON & SQL vs. NoSQL

1. Allgemeines

- Niklas – Prüfung am Papier (JSON-Files am Blatt Papier)
- Themen: Relational, NoSQL
- Voraussetzung: grobe DB-Kenntnisse

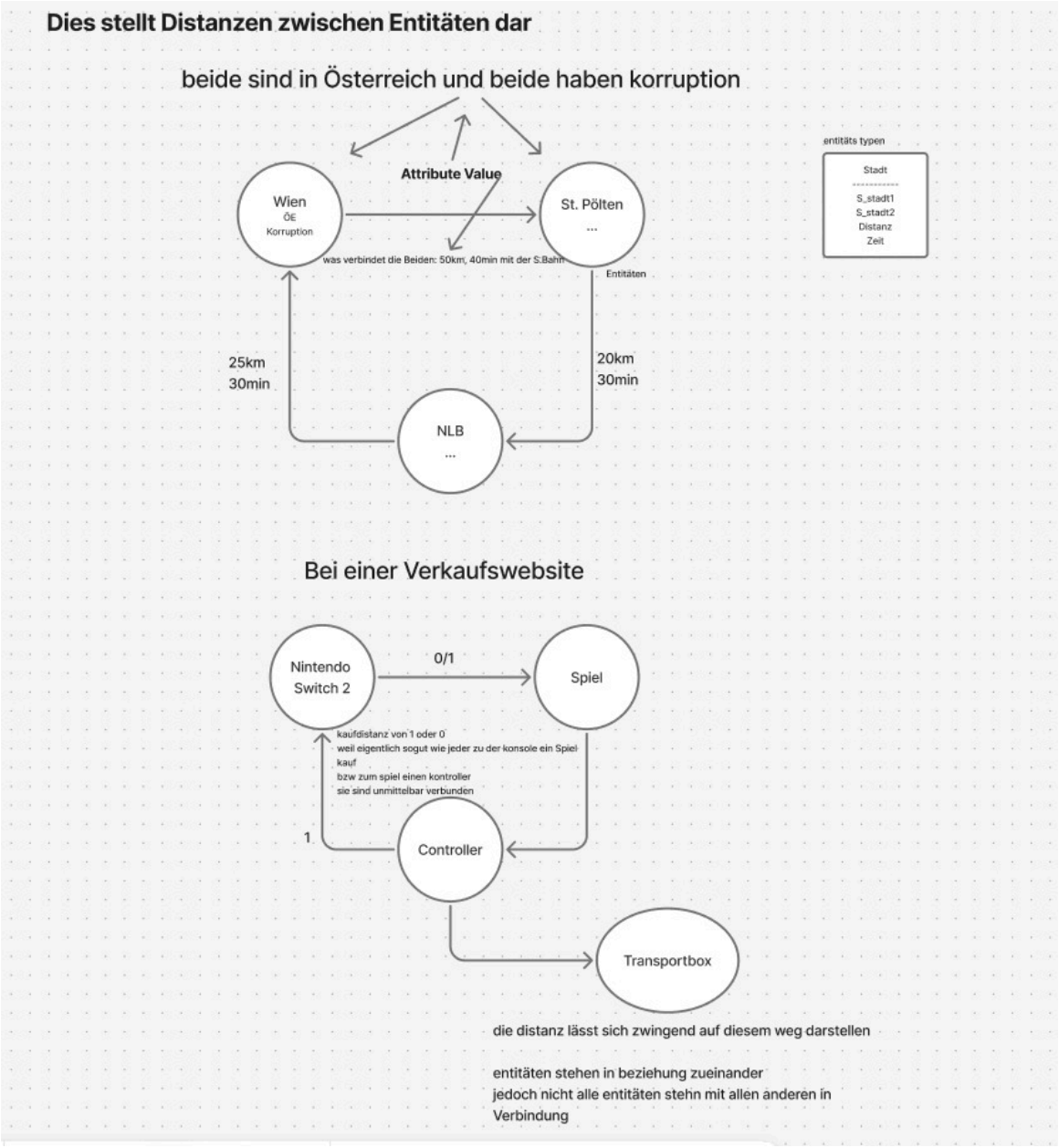


Abbildung 1: Distanzen zwischen Entitäten (Städte) und Kaufbeziehungen bei einer Verkaufswebsite

2. Gerichtete und ungerichtete Graphen

> **Prüfungsrelevant:** Unterschied zwischen gerichteten und ungerichteten Graphen.

Ungerichtet Die Beziehung gilt in beide Richtungen gleich.

Beispiel: Die Distanz Wien ⟶ St. Pölten ist dieselbe wie St. Pölten ⟶ Wien.

Gerichtet Die Beziehung hat eine Richtung, der Wert kann je Richtung verschieden sein.
Beispiel: „Wer eine Switch kauft, kauft auch einen Controller“ hat einen anderen Wert als „Wer einen Controller kauft, kauft auch eine Switch“.

3. Anwendung: Ähnlichkeiten erkennen

Graphen zeigen Ähnlichkeiten bzw. Zusammenhänge, z.B. bei Amazon: *Welche Produkte werden zusammen gekauft?*

3.1. Beispiel: Nintendo Switch

- Wer eine Switch kauft, kauft oft auch ein **Spiel** und einen **Controller**.
- Über zwei Schritte (Switch → Controller/Spiel → …) kommt man z.B. zu **Transportboxen**.
- Die Kanten tragen einen Faktor, z.B.: von 1000 Switch-Käufern kaufen 900 auch ein Spiel (0,9).

4. Technologien

- Graphdatenbanken (z.B. Neo4j) speichern Daten direkt als Knoten und Kanten.

5. JSON

Simples DB-Model fürs Anmelden von Reli:

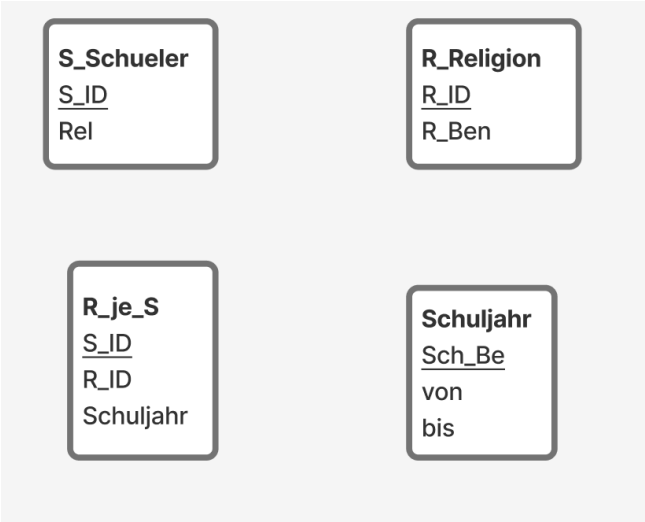


Abbildung 2: DB-Model Religionsanmeldung

5.1. CAP-Theorem

C Consistency
A Availability
P Partition

`[{}]` → eingebettetes Objekt in einem Array

5.2. Umwandlung vom DB-Model zu einem JSON

```
db.reljeSchuel.insertOne({
  name: "Neil",
  RelBezeichnung: …,
  Schuljahre: [{ Schuljahr: "26/27", gewählterUnterricht: …, Klasse: … }]
})
```

File wird umgewandelt – `SchuelerjeReligion` (`_id`: Religion):

```
db.SchuelerjeReligion.insertOne({
  Religion: …,
  Schuljahr: …,
```

```
  Schueler: [{
    id: …,
    schuelerfirstname: …,
    schuelerlastname: …,
    note: …            // (wahrscheinlich nicht gut)
  }],
  Lehrer: [{
    id: …,
    lehrername: …
  }]
})
```

Für jede Religion wird eine Liste an Schülern gespeichert und eine Liste von Lehrern, welche dieses Fach unterrichten. Für die Historisierung wird zusätzlich noch das Schuljahr gespeichert.

6. CRUD: SQL vs. NoSQL

CRUD – **C*reate,** R*ead, **U*pdate,** D*elete

	SQL	**NoSQL**
C	Langsam	Sehr schnell
R	Schnell	Langsam (nicht sehr schnell, kann aber bei kleinen sogar schneller sein als SQL wegen dem extra Join in SQL)
U	Mit Filter schnell Ohne Filter mittelschnell	Mit Filter langsam Ohne Filter leicht schnell
D	Mit Filter langsam Ohne Filter langsam	Mit Filter mittel Ohne Filter schnell – sehr schnell

Wir löschen alle Datensätze – warum braucht das bei SQL einen zweiten Prozess? ⟶ Es müssen Abhängigkeiten geprüft werden.

Eine DB hat eine DDL.

7. Schätzung Supermarkt

Wie viele Datensätze und Operationen gibt es?

Filialen ~1000, davon ~10 neu pro Jahr
Produkte ~40.000, ~1000 dazu pro Jahr, bis zu 2 Millionen Operationen pro Jahr
Rechnungen Datensätze pro Tag insgesamt (wie viele kaufen ein): ~10.000.000
Positionen min. 5 Produkte pro Rechnung: 50.000.000 insgesamt (deshalb wird viel gelesen [Read])

> **Prüfungsrelevant:**
> - Beurteilen, ob es ein leselastiger oder create-lastiger Prozess ist und welche DB man empfiehlt
> - aus dem E-Formular

8. Projekt

- Zweiergruppen
- Pre theoretisch (praktisch nicht unterrichtet)
- 4 Tabellen (1 Lookup und 2–3 Kreuztabellen)
- Neben der relationalen Schicht ein zweiter Ast, welcher auf eine GraphenDB (MongoDB) geht
- 3 verschiedene Skalierungen nehmen und Laufzeiten vergleichen (klein, mittel und groß – mindestens 1 Minute soll es dauern)

3. Dezember: Abgabe und Abgabegespräche (später gibt es Punkteabzug)